

OptiCrop – Smart Agricultural Production Optimization Engine

Project Documentation | Date: 5 July 2026 | Team ID: XXX

Project Description

OptiCrop is a machine-learning-based agricultural decision-support web application developed to recommend a suitable crop using seven soil and climate parameters: Nitrogen (N), Phosphorus (P), Potassium (K), temperature, humidity, soil pH, and rainfall.

The application uses a trained scikit-learn classification model stored as `model/crop_model.pkl`. A Flask backend loads the model, validates user inputs, generates the crop prediction, calculates the prediction confidence using class probabilities, and displays crop-specific information.

For every recommendation, OptiCrop can present the predicted crop, confidence percentage, water requirement, suitable soil type, fertilizer guidance, and a cultivation tip. The frontend is built with HTML, CSS, and JavaScript. The project is tested with Playwright, version-controlled on GitHub, and deployed publicly on Render.

Scenarios

Scenario 1

A farmer, student, or agricultural user enters the available N, P, K, temperature, humidity, pH, and rainfall values. OptiCrop analyzes the seven values and recommends a crop that best matches the supplied conditions.

Scenario 2

A user changes one or more soil or climate values and submits the form again. The application generates a new recommendation and confidence percentage, allowing the user to observe how different conditions influence the machine-learning prediction.

Scenario 3

After receiving the crop recommendation, the user reviews the associated water requirement, suitable soil, fertilizer guidance, and cultivation tip to better understand the recommended crop.

Technical Architecture

OptiCrop follows a web-based machine-learning architecture. The browser interface collects seven numerical inputs and submits them to the Flask backend through the `/predict` route. The backend validates the values, converts them into a NumPy feature array in the trained model's expected order, performs prediction using the serialized scikit-learn model, obtains class probabilities through `predict_proba()`, calculates the confidence percentage, retrieves crop-specific guidance, and renders the result back to the interface.

- Presentation Layer: HTML template, CSS styling, and JavaScript interaction.
- Application Layer: Python Flask application in `app/app.py`.
- Machine-Learning Layer: scikit-learn model loaded from `model/crop_model.pkl`.
- Data and Training Layer: `dataset/Crop_recommendation.csv` and notebook/`OptiCrop.ipynb`.
- Testing Layer: Playwright configuration, browser tests, test results, and HTML report.
- Deployment Layer: Render Web Service using Gunicorn.
- Version Control: Git and GitHub.

Pre-requisites

- Python 3.x and pip.
- Basic Python and Flask knowledge.
- Basic machine-learning and scikit-learn knowledge.
- HTML, CSS, and JavaScript fundamentals.
- Git and GitHub.
- VS Code or another development environment.
- A modern web browser.
- Basic Render deployment knowledge.

Project Workflow

Milestone 1: Requirement Analysis and Planning

- Identify the agricultural problem to be addressed through crop recommendation.
- Define the seven input parameters: N, P, K, temperature, humidity, pH, and rainfall.
- Study the crop recommendation dataset and determine the target crop labels.
- Finalize the technology stack and application architecture.

Milestone 2: Data Preparation and Model Development

- Use Crop_recommendation.csv as the project dataset.
- Explore and prepare the seven input features and crop target labels.
- Train and evaluate the crop classification model using scikit-learn.
- Save the trained model as model/crop_model.pkl for use by the Flask application.

Milestone 3: Backend Development

- Develop the Flask application in app/app.py.
- Load the trained model using a relative path that works locally and on Render.
- Create the home route for the main interface.
- Create the POST /predict route for recommendation requests.
- Validate pH, humidity, temperature, rainfall, and NPK values.
- Generate the crop prediction and confidence percentage.
- Retrieve crop-specific water, soil, fertilizer, and cultivation guidance.

Milestone 4: Frontend Development

- Create the main interface in app/templates/index.html.
- Create the visual styling in app/static/style.css.
- Use app/static/script.js for client-side interaction and validation support.
- Provide seven clearly labeled input fields.
- Display the recommendation, confidence, and crop guidance in the result section.

Milestone 5: Testing and Deployment

- Perform manual functional testing using multiple valid input combinations.
- Test input validation and error handling.
- Use Playwright for automated browser testing.
- Maintain the Playwright HTML report and test-result artifacts.
- Deploy the Flask application to Render using Gunicorn.
- Verify the publicly deployed application and prediction workflow.

Milestone 6: Project Completion and Documentation

- Organize the application, model, dataset, notebook, tests, reports, and documentation.
- Maintain the GitHub repository and README.
- Prepare project documentation and demonstration material.
- Provide the live deployment URL, repository URL, and demo video.

Milestone 1: Requirement Gathering and Analysis

Activity 1.1: Identify the Problem and Project Objective

The project addresses the need for a simple decision-support system that can use measurable soil nutrient values and environmental conditions to recommend a suitable crop. The objective is to convert seven numerical inputs into a crop recommendation through a trained machine-learning classification model.

Activity 1.2: Study the Dataset and Input Features

The project uses dataset/Crop_recommendation.csv. The input features used by the application are Nitrogen, Phosphorus, Potassium, temperature, humidity, soil pH, and rainfall. The target is the crop label. The dataset and training workflow support the model used by the deployed Flask application.

Activity 1.3: Define the Application Architecture

The application is divided into a browser-based frontend, Flask backend, trained machine-learning model, supporting dataset and notebook, automated browser tests, and deployment configuration. The user

submits input values through the frontend, Flask validates and processes the request, and the trained model generates the recommendation.

Activity 1.4: Set Up the Development Environment

- Install Python 3.x and pip.
- Create a virtual environment using: `python -m venv venv`
- Activate it on Windows using: `venv\Scripts\activate`
- Install dependencies using: `pip install -r requirements.txt`
- Run the local application using: `python app/app.py`
- Open the local address shown by Flask, normally `http://127.0.0.1:5000/`.

Milestone 2: Core Functionalities Development

Activity 2.1: Develop the Core Recommendation Workflow

- Collect N, P, K, temperature, humidity, pH, and rainfall from the user.
- Validate the submitted values before prediction.
- Convert valid values into the numerical array expected by the model.
- Predict the crop using `model.predict()`.
- Calculate confidence from `model.predict_proba()`.
- Display the crop recommendation and supporting crop information.

Activity 2.2: Implement Input Validation

- Soil pH must be between 0 and 14.
- Humidity must be between 0 and 100 percent.
- Temperature must be between 0°C and 60°C.
- Rainfall cannot be negative.
- N, P, and K values cannot be negative.
- Invalid input returns a clear error message on the application page.

Milestone 3: app.py Development

The `app/app.py` file contains the core backend logic. It creates the Flask application, constructs the deployment-safe model path, loads `crop_model.pkl`, defines crop information, serves the home page, receives prediction requests, validates inputs, performs model inference, calculates confidence, and renders the result.

Activity 3.1: Model Loading

`BASE_DIR` is calculated from the location of `app.py`. `MODEL_PATH` then points from the app folder to `../model/crop_model.pkl`. This allows the model to be found correctly in the current project structure during local execution and Render deployment.

Activity 3.2: Prediction Processing

The `/predict` route reads the seven submitted form values as floating-point numbers. After validation, the values are placed in a NumPy array in the order N, P, K, temperature, humidity, pH, and rainfall. The model predicts the crop, while class probabilities are used to calculate the confidence percentage.

Activity 3.3: Crop Information

The `crop_info` dictionary stores supporting information for the crops handled by the application. For the predicted crop, OptiCrop displays water requirement, suitable soil, fertilizer guidance, and a cultivation tip. A default information block is available if a predicted label does not have a specific entry.

Milestone 4: Frontend Development

The frontend is designed as a clear single-page interface for entering the seven agricultural parameters and viewing the machine-learning recommendation. The interface uses HTML for structure, CSS for presentation, and JavaScript for browser-side interaction.

Activity 4.1: User Interface

- Input fields for Nitrogen, Phosphorus, Potassium, temperature, humidity, soil pH, and rainfall.
- Analyze & Recommend action for submitting the values.
- Result area for the recommended crop and confidence.
- Supporting cards for water requirement, suitable soil, fertilizer guidance, and cultivation tip.

Activity 4.2: Actual OptiCrop Interface and Results

OptiCrop AI
Smart Agricultural Production Optimization Engine

Soil & Climate Parameters

Nitrogen (N)
Example: 90

Phosphorus (P)
Example: 42

Potassium (K)
Example: 43

Temperature (°C)
Example: 25.5

Humidity (%)
Example: 82

AI Analysis Waiting...
Enter the soil and environmental parameters on the left.
Click **Analyze & Recommend** to receive an AI-powered...

Figure 1: Actual OptiCrop input interface supplied from the working project.

Soil & Climate Parameters

Nitrogen (N)
50.0

Phosphorus (P)
50.0

Potassium (K)
50.0

Temperature (°C)
30.0

Humidity (%)
52.0

Soil pH
6.0

Rainfall (mm)
100.0

Analyze & Recommend

AI Recommendation
mango
Confidence **41.0 %**

Water Requirement
Moderate

Suitable Soil
Deep Loamy Soil

Fertilizer
Organic Compost

Cultivation Tip
Avoid waterlogging.

Figure 2: Actual OptiCrop result showing a Mango recommendation with confidence and crop guidance.

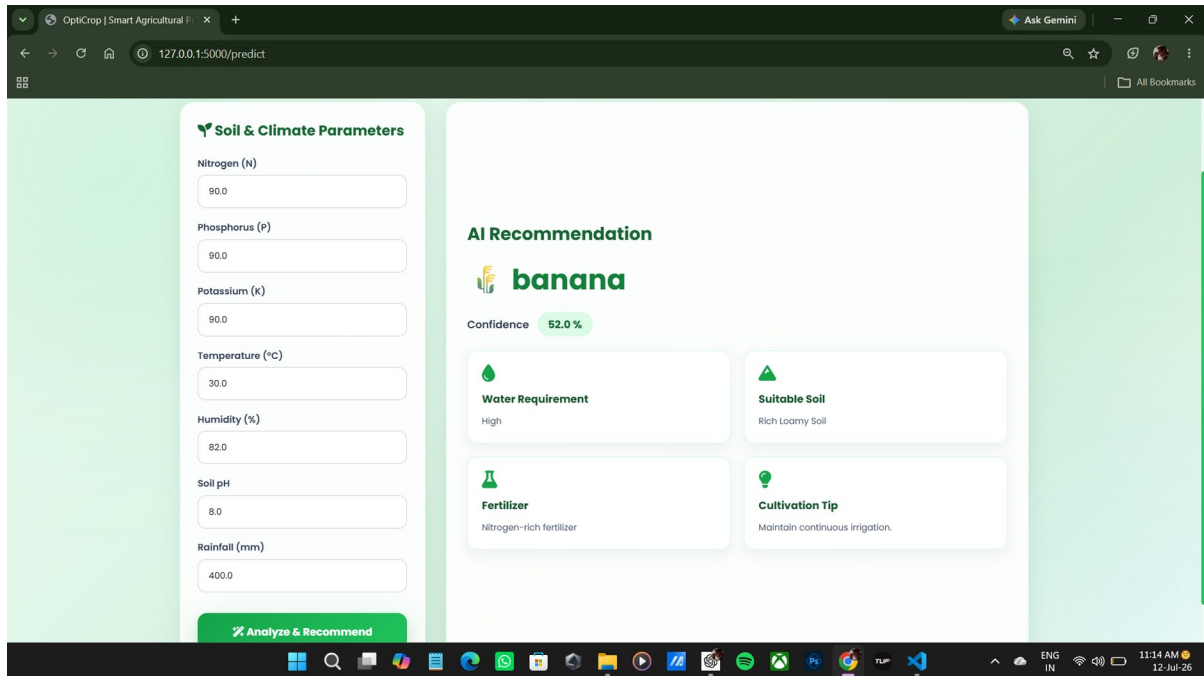


Figure 3: Actual OptiCrop result showing a Banana recommendation with confidence and crop guidance.

Project File / Folder Structure

The project repository contains the Flask application, static and template files, crop dataset, trained model, Jupyter notebook, Playwright tests and reports, project documentation, dependency files, Procfile, README, and Git configuration.

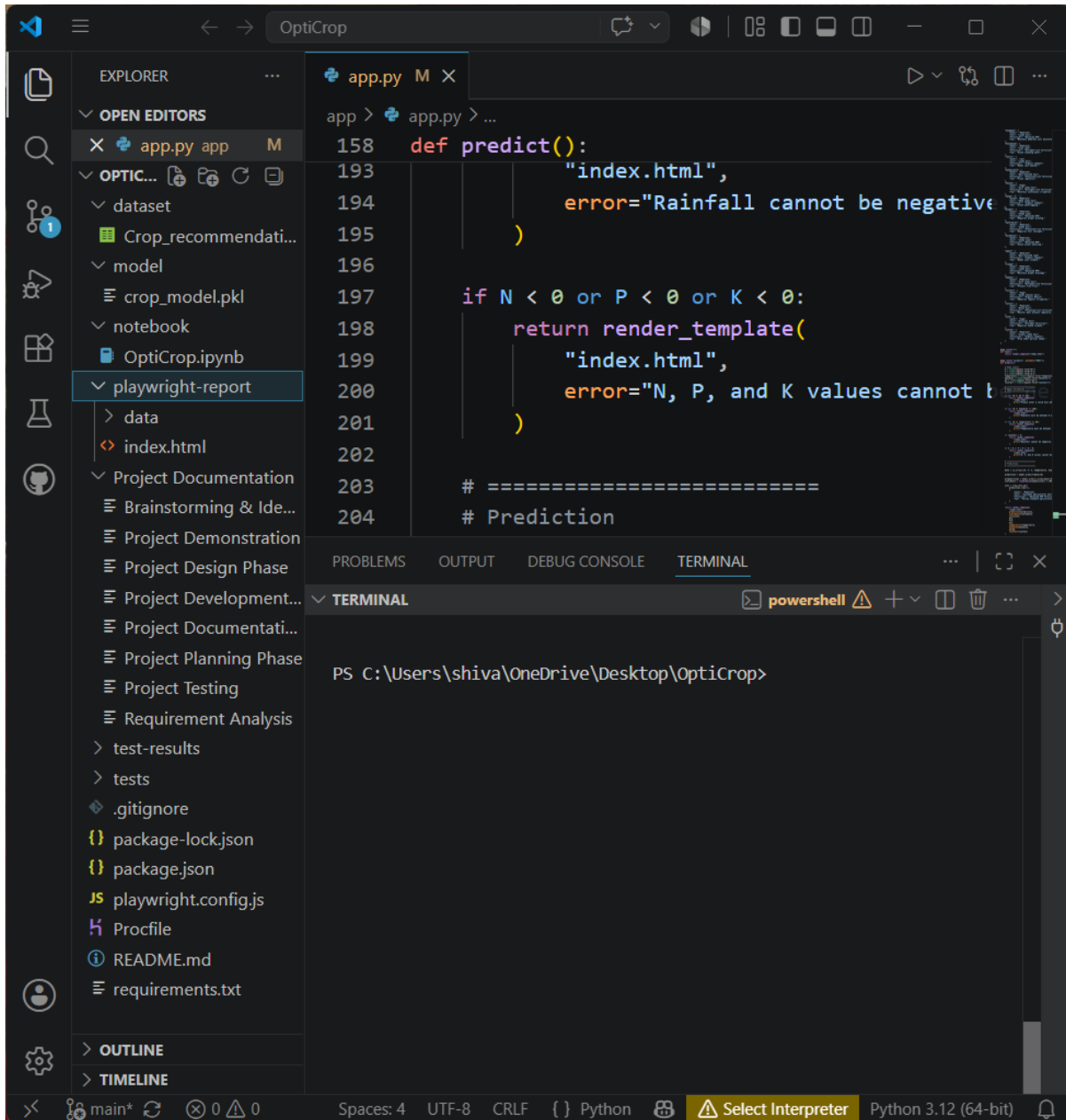


Figure 4: Actual OptiCrop project structure supplied from the project folder.

Milestone 5: Testing and Deployment

Activity 5.1: Application Testing

- Manual functional testing of the input and prediction workflow.
- Validation testing for pH, humidity, temperature, rainfall, and NPK values.
- Browser automation using Playwright.
- Cross-browser test execution for Chromium, Firefox, and WebKit.
- Test artifacts stored in test-results.
- Generated Playwright HTML report stored in playwright-report.

Activity 5.2: Deployment on Render

- Maintain the source code in the GitHub repository.
- Use requirements.txt for Python dependencies.
- Use Gunicorn as the production WSGI server.
- Use the production start command: `gunicorn --chdir app app:app`
- Deploy the main branch as a Render Web Service.

- Verify the public application after deployment.

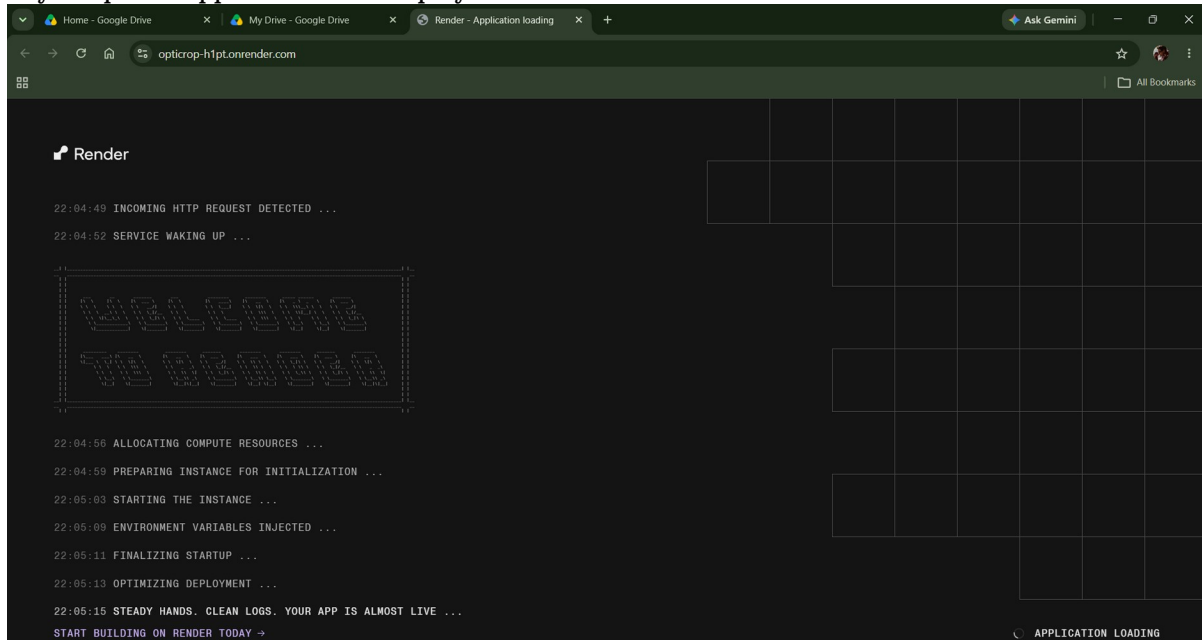


Figure 5: Actual Render startup screen supplied for the deployed OptiCrop application.

Deployment Details

- Hosting Platform: Render.
- Backend Framework: Flask.
- Machine-Learning Library: scikit-learn.
- Production Server: Gunicorn.
- Version Control: GitHub.
- Login Requirement: No login required.

Project URL

OptiCrop Deployed Application: <https://opticrop-h1pt.onrender.com/>

OptiCrop GitHub Repository: <https://github.com/sivaeeduri01/OptiCrop-Smart-Agricultural-Production-Optimization-Engine>

OptiCrop Demo Video: <https://drive.google.com/file/d/1j21qATbv5oR1h6nIJ24FIhlu0EGnVB/view?usp=sharing>

Descriptions of Each Module

Soil and Climate Input Module

Collects the seven numerical values required by the trained model: N, P, K, temperature, humidity, pH, and rainfall.

Validation Module

Checks whether the submitted values satisfy the limits implemented in app.py and returns clear error messages when values are invalid.

Prediction Module

Converts validated input into the expected NumPy feature array and uses the trained scikit-learn model to predict the crop class.

Confidence Module

Uses the model's class probabilities to calculate and display the confidence percentage for the recommendation.

Crop Information Module

Maps the predicted crop to practical information including water requirement, suitable soil, fertilizer guidance, and a cultivation tip.

Frontend Module

Provides the browser interface using index.html, style.css, and script.js.

Testing Module

Contains Playwright tests, Playwright configuration, test results, and the generated HTML test report.

Deployment Module

Uses requirements.txt, Procfile/start command configuration, GitHub source control, Render hosting, and Gunicorn to serve the Flask application publicly.

Conclusion

OptiCrop demonstrates an end-to-end machine-learning web application for agricultural decision support. It combines a crop recommendation dataset, model development, a serialized scikit-learn model, Flask backend logic, an HTML/CSS/JavaScript interface, input validation, confidence calculation, crop guidance, Playwright testing, GitHub version control, and Render deployment. The final system accepts seven soil and climate parameters and provides a crop recommendation with supporting information through a publicly accessible web application.

Prepared by: Siva Eeduri